

2024 年非专业级别软件能力认证第一轮

(CSP-J) 入门级 C++语言模拟试题

考生注意事项:

试题纸共有 9 页, 答题纸共有 1 页, 满分 90 分。请在答题纸上作答, 写在试题纸上的一律无效。

不得使用任何电子设备(如计算器、手机、电子词典等)或查阅任何书籍资料。

一、单项选择题(共 15 题, 每题 2 分, 共计 30 分; 每题有且仅有一个正确选项)

1、计算机主机是由 CPU 与(D)构成的。

- A. 控制器 B. 运算器 C. 输入、输出设备 D. 内存储器

2、算式 $(2047)_{10} - (3FF)_{16} + (2000)_8$ 的结果是(A)。

- A. (2048)₁₀ B. (2049)₁₀ C. (3746)₈ D. (1AF7)₁₆

3、ASCII 码的含义是(B)。

- A. 二、十进制转换码 B. 美国信息交换标准代码
C. 数字的二进制编码 D. 计算机可处理字符的唯一编码

4、已知 x 的原码表示为 11110111, 下列(B)是 x 的补码表示。

- A. $[x]_{\text{补}} = 01010011$ B B. $[x]_{\text{补}} = 10001001$ B
B. $[x]_{\text{补}} = 11111111$ B D. $[x]_{\text{补}} = 11000000$ B

5、在编程时(使用任一种高级语言, 不一定是 C 语言), 如果需要从磁盘文件中输入一个很大的二维数组(例如 1000×1000 的 double 型数组), 按行读(即外层循环是关于行的)与按列读(即外层循环是关于列的)相比, 在输入效率上(D)

- A. 没有区别 B. 按行读的方式要高一些
B. 按列读的方式要高一些 D. 取决于数组的存储方式

6、某数列有 1000 个各不相同的单元, 由低至高按序排列。现要对该数列进行二分法检索(binarysearch), 在最坏的情况下, 须检视(B)个单元。

- A. 1000 B. 10 C. 100 D. 500

7、在 C 语言程序中,判断 a 等于 0 或 b 等于 0 或 c 等于 0 的正确的条件表达式是 (B)。

- A. $!((a!=0)(b!=0)|(c)!=0)$ B. $!((a!=0)\&\&(b!=0)\&\&(c)!=0)$
B. $!(a==0\&\&b==0)|(c)!=0$ D. $(a=0)\&\&(b=0)\&\&(c)=0$

8、已知队列 (13, 2, 11, 34, 41, 77, 5, 7, 18, 26, 15), 第一个进入队列的元素是 13, 则第五个出队列的元素是 (B)

- A. 5 B. 41 C. 77 D. 13

9、有 5 个不同的球和 3 个不同的盒子, 每个盒子至少放一个球, 有 (B) 种不同的放法?

- A. 60 B. 120 C. 150 D. 180

10、从 5 个不同的球中取出 2 个, 不考虑顺序, 有 (A) 种不同的组合?

- A. 10 B. 12 C. 15 D. 20

11、假设 A=true, B=false, C=true, D=true, 逻辑运算表达式 $A \wedge B \vee C \wedge D$ 的值是 (A)

- A. True B. false C. 0 D. 1

12、以下哪一个不是栈的基本运算 (B)

- A. 删除栈顶元素 B. 删除栈底的元素
C. 判断栈是否为空 D. 将栈置为空栈

13. 一个二叉树的前序遍历是 HGBDAFEC, 中序遍历是 BGHFAEDC, 同时采用顺序存储结构, 即用一维数组元素存储该二叉树中的结点 (根结点的下标为 1, 若某结点的下标为 i, 则其左孩子位于下标 $2i$ 处、右孩子位于下标 $2i+1$ 处), 则该数组的最大下标至少为 (B)

- A. 7 B. 13 C. 15 D. 12

14. 对于“树”这种数据结构, 正确的有: (C)

- ①一个有 n 个顶点、n-1 条边的图是树
②一个树中的两个顶点之间有且只有一条简单路径
③树中一定存在度数不大于 1 的顶点

④树可能存在环

A. ①②④

B. ①②③

C. ②③

D. ①②

15. 在一个具有 n 个顶点的无向图中，要连通全部顶点至少需要 (C) 条边。

A. n

B. $n + 1$

C. $n - 1$

D. $n/2$

二、阅读程序（程序输入不超过数组或字符串定义范围；判断题正确填√，错误填×；出特殊说明外，判断题 2 分，选择题 3 分，合计 30 分）

(1)

```
01 #include <iostream>
02 using namespace std;
03
04 int main() {
05     int n, count = 0, flage = 0, max = 0, t;
06     scanf("%d", &n);
07     t = n;
08     for (int i = 2; i <= n; i++) {
09         count = 0;
10         if (t % i == 0) {
11             while (t % i == 0) {
12                 t = t / i;
13                 count++;
14             }
15         }
16         if (max < count) {
17             flage = i;
18             max = count;
19         }
20         if (t == 1) {
21             break;
22         }
23     }
24     printf("%d %d", flage, max);
25     return 0;
26 }
```

假设输入所有数均为不超过 10^9 的数，完成下面的判断题和单选题

判断题

16. 删除第 7 行程序行为不变。(☒)
17. 第 9 行的 `count=0`，可以编写在 `for` 循环之外。(☒)
18. 第 20、21 行删除对我的程序输出结果没有影响。(☒)
19. 当程序输入 16 时，输出为 2 4。(☒)

单选题

20. 当输入为 24 时输出为 (☒ A)。
- A. 2 3 B. 1 3 C. 8 1 D. 3 2

本程序为求解幂次最高的因子，`count` 是记录当前因子的次幂，`max` 记录最高次幂，`flage` 记录最高次幂因子。

- 16 第三题第 7 行后无法判断因子，会导致程序行为异常故错误。
- 17 `count` 变量每一个新的因子均需要单独记数，不能放在循环之外，故错误。
- 18 20、21 行为特判，提前结束循环，防止程序超时，故对程序输出没有影响。
- 19 输入为 16 时，可以分解为 $16 = 2 \times 2 \times 2 \times 2$ ，因子为 2，最高次为 4，故正确。
- 20 输入为 24 时可以分解为 $24 = 2 \times 2 \times 2 \times 3$ ，因子为 2 和 3，幂次最高的因子为 2，幂次为 3，故选 A。

(2)

```
01 #include <iostream>
02 using namespace std;
03 typedef long long ll;
04 long long s[100005];
05 int v[100005], q[100005];
06
07 int main() {
08     int n, d;
09     cin >> n >> d;
10     s[1] = 0;
11     for (int i = 2; i <= n; i++) {
12         cin >> s[i];
13         s[i] += s[i - 1];
14     }
15     for (int i = 1; i <= n; i++) {
```

```

16         cin >> v[i];
17     }
18     int cnt = 1;
19     q[1] = 1;
20     for (int i = 2; i <= n; i++) {
21         if (v[i] < v[q[cnt]]) {
22             cnt++;
23             q[cnt] = i;
24         }
25     }
26     if (q[cnt] != n) {
27         cnt++;
28         q[cnt] = n;
29     }
30     ll now = 0, ans = 0;
31     for (int i = 1; i < cnt; i++) {
32         ll t = s[q[i + 1]] - s[q[i]];
33         if (t < now) {
34             now -= t;
35         } else {
36             t -= now;
37             ll tmp = (t + d - 1) / d;
38             ans += v[q[i]] * tmp;
39             now = d * tmp - t;
40         }
41     }
42     cout << ans;
43     return 0;
44 }

```

假设输入的所有数据均不超过 10^5 ，完成下面习题：

判断题

21. 把 13 行改为 `s[i] = s[i] + s[i-1]` 对程序没有影响。(☒)
22. 第三行的 `typedef long long ll`; 是将 `long long` 这个长整形重命名为 `ll`。
(☒)

23. 所有的输入均不超过 10^5 ，该程序可以不使用 long long 数据类型，对程序的输出结果没有影响。(×)

单选题

24. 程序中 s 数组的用途是什么 (B)。

- A. 存储每个位置的值
- B. 存储前缀和计算总路程
- C. 存储值的最大值
- D. 存储位置索引

25. 程序的时间复杂度为 (C)。

- A. $O(n^2)$
- B. $O(n\log n)$
- C. $O(n)$
- D. $O(n^2\log n)$

26. 程序输出的结果 ans 的含义是什么 (C)。

- A. ans 是数组 v 中元素的总和
- B. ans 是所有 $s[q[i + 1]] - s[q[i]]$ 的总和
- C. ans 是根据 v 数组和 s 数组计算的加权总和
- D. ans 是 d 和 v 数组中的最大值的乘积

答案解析：本程序为 2023 年 CSP-J2 T2(公路)使用前缀和计算出需要行驶的总路程，v 数组为每个加油站的单价，循环查找单价最小的并将索引记录在 q 数组中，最后遍历 q 数组，判断当前的油是否够走到下一站单价小的加油站，如果够就不加，不够就需要加，刚好到单价低的加油站（向上取整），最后加权和存储在 ans 中。

21. 复合运算，正确

22. 基础知识，正确

23. 程序 38 行为乘法运算相乘结果可能为 10^{10} 超过 int 存储范围，表述错误

24. 前缀和

25. 所有主要的操作都在 $O(n)$ 的时间复杂度内，因此整个程序的时间复杂度是 $O(n)$

26. 单价和数量的加权总和，总价钱

三、完善程序（单选题，每小题 3 分、合计 30 分）

(1)(最小公倍数)使用辗转相除法求解最大公约数，在计算出最小公倍数。(输入数据保证 $1 \leq a, b \leq 10^6$, 且前一个小于后一个)

```

01 #include <iostream>
02 using namespace std;
03
04 int gcd(int a, int b) {
05     while (①) {
06         int temp = b;
07         ②;
08         ③;
09     }
10     return a;
11 }
12
13 int lcm(int a, int b) {
14     return a * b / gcd(a, b)④;
15 }
16
17 int main() {
18     int n;
19     cin >> n;
20     for (int i = 0; i < n; i++) {
21         int a, b;
22         cin >> a >> b;
23         cout <<⑤ << endl;
24     }
25     return 0;
26 }

```

27. ①处应填 (**B**)

A. $a \neq 0$ **B. $b \neq 0$** C. $a \neq 0 \ || \ b \neq 0$ D. $a \neq 0 \ \&\& \ b \neq 0$

28. ②处应填 (**B**)

A. $a = b \% a$ **B. $b = a \% b$** C. $a = b / a$ D. $b = a / b$

29. ③处应填 (**A**)

A. $a = temp$ B. $b = temp$ C. $temp \% = a$ D. $temp \% = b;$

30. ④处应填 (**A**)

A. $a * b / gcd(a, b)$ B. $b * gcd(a, b)$

D. $a / b * \text{gcd}(a, b)$

A. $\text{lcm}(a, b)$

C. $\text{lcm}(a, b) * \text{gcd}(a, b)$

D. $\text{lcm}(a, b) / \text{gcd}(a, b)$

```

01 #include<iostream>
02 #include<vector>
03 using namespace std;
04 const int MAXN = 1e5+1;
05 int n, m, S , T, deg[MAXN];
06 vector<int> E[MAXN];
07 long long f[MAXN];
08 struct block {
09     int x, y;
10 };
11 vector<block> list;
12 int main() {
13     cin >> n >> m >> S >> T;
14     for (int i = 0; i < m; ++i) {
15         int u, v;
16         cin >> u >> v;
17         E[u].push_back(v);
18         ++deg[v];
19     }
20     vector<int> Q;
21     for (int i = 1; i <= n; ++i)
22         if (!deg[i]) Q.push_back(i);
23     ① {
24         int u = Q[i];
25         for (int v : E[u]) {
26             if (②) Q.push_back(v);

```



```

27         --deg[v];
28     }
29 }
30 reverse(Q.begin(), Q.end());
31 ③;
32 for (int u : Q) {
33     for (int v : E[u]) ④;
34 }
35 cout << ⑤ << endl;
36 return 0;
37 }

```

32. ①处应填(A)

- A. `for (int i = 0; i < n; ++i)`
- B. `for (int i = 0; i <= n; ++i)`
- C. `for (int i = 1; i < n; ++i)`
- D. `for (int i = 1; i <= n; ++i)`

33. ②处应填(C)

- A. `deg[v] == 0`
- B. `deg[v] > 0`
- C. `deg[v] == 1`
- D. `deg[v] > 1`

34. ③处应填(B)

- A. `f[T] = 0`
- B. `f[T] = 1`
- C. `f[S] = 0`
- D. `f[S] = 1`

35. ④处应填(A)

- A. `f[u] += f[v]`
- B. `f[v] += f[u]`
- C. `f[u] *= f[v]`
- D. `f[v] *= f[u]`

36. ⑤处应填(B)

- A. $f[1]$ B. $f[S]$
C. $f[T]$ D. $f[n]$

先拓扑排序求出拓扑序，然后从终点倒退回起点

1. A 从 S 点出发把所有路径数都求出来，所以循环次数是 n, 点是从 Q 这个 vector 取的，vector 下标从 0 开始
2. C 因为代码中是先判断是否要加入队列，再入度-1。
3. B 初始相当于是自己出发到达自己，路径数 1（30 行可知是倒着求的所以是终点赋值）
4. A 是方案之和
5. B 是倒着求的所以输出的下标是起点